
University of Kansas

**<Project Name>
Software Requirements Specifications**

Version 1.0

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

Revision History

Date	Version	Description	Author
03/Mar/26	0.1	Added content for sections 1.5 and 3.3.	John P.
7/Mar/26	0.2	Added content for section 3.1	Austin Haight
7/Mar/26	0.3	Added content for sections 1.1 and 3.2	Yecolia E.
10/Mar/26	0.4	Added content for sections 4 and 5	Tugs-Oyut T.
10/Mar/26	0.5	Added content for section 1.2	Eric F.
11/Mar/26	0.6	Added content for sections 2.*	Alexander Wichern

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

Table of Contents

1.	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions, Acronyms, and Abbreviations	4
1.4	References	4
1.5	Overview	4
2.	Overall Description	6
2.1	Product perspective	6
2.1.1	System Interfaces	6
2.1.2	User Interfaces	6
2.1.3	Hardware Interfaces	6
2.1.4	Software Interfaces	6
2.1.5	Communication Interfaces	6
2.1.6	Memory Constraints	6
2.1.7	Operations	6
2.2	Product functions	6
2.3	User characteristics	6
2.4	Constraints	6
2.5	Assumptions and dependencies	6
2.6	Requirements subsets	6
3.	Specific Requirements	6
3.1	Functionality	6
3.1.1	<Functional Requirement One>	7
3.2	Use-Case Specifications	7
3.3	Supplementary Requirements	7
3.3.1	Usability	7
3.3.2	Performance	7
3.3.3	Reliability	7
3.3.4	Portability	7
3.3.5	Constraints	7
4.	Classification of Functional Requirements	7
5.	Appendices	8

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

Software Requirements Specifications

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a detailed description of the requirements for the Arithmetic Expression Evaluator system implemented in C++. This document outlines the system's functional and non-functional requirements, including its behavior, constraints, and expected performance.

The SRS serves as a reference for developers, testers, and project stakeholders by defining how the system should behave when parsing and evaluating arithmetic expressions. It ensures that all team members have a shared understanding of the system requirements before design and implementation begin. Additionally, this document provides a foundation for validating that the final implementation meets the intended system functionality.

1.2 Scope

This Software Requirements Specification (SRS) describes the requirements for a **C++ Arithmetic Expression Evaluator**. The application allows users to input arithmetic expressions containing numbers, operators (+, -, *, /), and parentheses, and computes the correct result according to standard order of operations. The system will process user input, evaluate the expression, and display the calculated output through a command-line interface. This document defines the functional behavior of the system, including expression parsing, evaluation, and basic error handling for invalid input. The system is limited to evaluating basic arithmetic expressions and does not include advanced mathematical functions or graphical interfaces.

1.3 Definitions, Acronyms, and Abbreviations

[This subsection provides the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS. This information may be provided by reference to the project's Glossary.]

1.4 References

N/A

1.5 Overview

The SRS describes what the arithmetic expression parser should accomplish, including its key features, services, and performance objectives. It identifies and documents the purpose of the arithmetic expression evaluator and specifies the capabilities the software must provide, as well as the services it will deliver.

This document is organized to provide a structured understanding of the expression evaluator and its requirements:

- **Section 1 – Introduction:** Establishes the purpose, scope, key terms, references, and context of the SRS using subsections for each.
- **Section 2 – Overall Description:** Provides background information and general details to support understanding of the software requirements, including product perspective, product functions, user characteristics, constraints, and assumptions.
- **Section 3 – Specific Requirements:** Details use-case specifications, functional and non-functional requirements, and constraints for the arithmetic parser software using natural-language descriptions.

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

- **Section 4 – Classification of Functional Requirements:** Presents a table of all functional requirements, categorized by type or order of appearance.
- **Section 5 – Appendices:** Includes references and supporting materials specifying which appendices are part of the requirements.

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

2. Overall Description

2.1 Product perspective

2.1.1 User Interfaces

The user interface will be text-based command line prompts. Returning calculated values on valid expression and returning warnings and/or error messages in case of invalid input or ambiguity.

2.1.2 Software Interfaces

Interacts with the C++ standard library.

2.1.3 Memory Constraints

None specified, don't overflow the stack on nested parentheses.

2.2 Product functions

The software will tokenize user input and evaluate user expressions. This includes maintaining correct order of operations and handling precedence in the case of parentheses (PEMDAS). The software will also handle unary operators (signage and inc/decrementing) as well as error handling on invalid or ambiguous inputs.

2.3 User characteristics

Users are anyone who has a basic understanding of mathematics.

2.4 Constraints

Must be written in C++, utilize object-oriented design, have unit test to verify correctness, ensure that the program provides clear and informative messages upon invalid input

2.5 Assumptions and dependencies

Inputs are signed integers or floats

3. Specific Requirements

3.1 Functionality

3.1.1 User Interface

The program will be interacted with via a command line or shell. The program must have a detailed man page and clearly state errors to the user.

3.1.2 History/Memory

The program should keep a number of past expressions from the current session, and make them available to the user. The user should be able to easily pull values from past expressions to use in the current one.

3.1.3 Token Parsing

The program must be able to turn any syntactically valid string into an ordered list of token objects, which can be evaluated mathematically. Tokens may be separated by an arbitrary number of space characters. Tokens must be evaluated in PEMDAS order. Tokens include:

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

-Integer: Any length of digits, must not contain a period. May have a sign prefix but otherwise is assumed positive valued. Regex: $[+-]?[0-9][0-9]^*$

-Floating Point: Any two lengths of digits $[0-9]$ with exactly one period. May have a sign prefix but otherwise is assumed positive valued. Regex: $[+-]?([0-9][0-9]^*\.[0-9]^*|([0-9]^*\.[0-9][0-9]^*))$

-Binary Addition: Two expressions separated by a '+' operator. Adds the results of the two expressions. Resolves to an Integer only if both operands are Integers, otherwise Floating Point.

-Binary Subtraction: Two expressions separated by a '-' operator. Subtracts the result of the second expression from the result of the first expression. Resolves to an Integer only if both operands are Integers, otherwise Floating Point.

-Binary Multiplication: Two expressions separated by a '*' operator. Multiplies the results of the expressions. Resolves to an Integer only if both operands are Integers, otherwise Floating Point.

-Binary Division: Two expressions separated by a '/' operator. Divides the result of the second expression from the result of the first expression. Resolves to a Floating Point.

-Binary Floor Division: Two expressions separated by a '//' operator. Performs Binary Division and then truncates the result into an Integer.

-Modulus: Two expressions separated by a '%' operator. For input $A\%B$ the result is equivalent to $A - ((A/B) * A)$. Resolves to an Integer only if both operands are Integers, otherwise Floating Point.

-Exponentiation: Any expression followed by a '**' operator and an Integer-valued expression. Multiplies the result of the first expression by itself a number of times equal to the result of the second expression. Resolves to the type of the first expression.

-Parentheses: Any expression enclosed by parentheses '(', ')'. Forces priority in evaluation.

3.2 Use-Case Specifications

Use Case 1 -- Evaluate Arithmetic Expression

Use Case Name: Evaluate Arithmetic Expression

Actor: User

Description:

The user enters an arithmetic expression, and the system evaluates the expression according to operator precedence and mathematical rules.

Preconditions:

- The program is running.
- The user has access to the command-line interface.

Main Flow:

1. The user enters an arithmetic expression.
2. The system parses the input expression.
3. The system identifies numbers, operators, and parentheses.
4. The system applies operator precedence rules.
5. The system evaluates the expression.
6. The result is displayed to the user.

Postconditions:

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

- The correct result of the expression is displayed.

Example:

Input: $3 + 4 * 2$

Output: 11

Use Case 2 -- Handle Invalid Expression

Use Case Name: Handle Invalid Expression

Actor: User

Description:

The system detects and reports errors when the user enters an invalid arithmetic expression.

Preconditions:

- The program is running.

Main Flow:

1. The user enters an arithmetic expression.
2. The system analyzes the expression.
3. The system detects an error (such as division by zero, invalid characters, or unmatched parentheses).
4. The system displays an appropriate error message.

Postconditions:

- The user is informed that the expression is invalid.

Example:

Input: $5 / 0$

Output: Error: Division by zero

Use Case 3 -- Evaluate Expression with Parentheses

Use Case Name: Evaluate Expression with Parentheses

Actor: User

Description:

The system evaluates expressions containing parentheses to ensure the correct order of operations.

Main Flow:

1. The user enters an expression containing parentheses.
2. The system evaluates expressions inside parentheses first.
3. The system applies remaining operations according to precedence rules.
4. The final result is displayed.

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

Example:

Input: $8 - (5 - 2)$

Output: 5

3.3 Supplementary Requirements

This section establishes requirements that create constraints on the functional behavior of the arithmetic expression evaluator. These include non-functional requirements and development constraints.

3.3.1 Usability

The program must provide a command-line interface that clearly prompts the user to enter an arithmetic expression. After evaluation, the program must display the result in a legible format that clearly highlights the input expression and the calculated output. All prompts and outputs must follow a consistent format using clear and unambiguous language.

3.3.2 Performance

The parser must evaluate expressions efficiently without a noticeable delay in the time to evaluate. In addition, the software should allow consecutive calculations without a restart.

3.3.3 Reliability

The parser must handle errors, including invalid expressions or division by zero, that display the error in a user-friendly way. All solutions must be accurate and clearly inform the user of the intended result.

3.3.4 Portability

The parser must compile and run on different platforms or environments, such as Windows, Linux, and macOS. Users across all platforms must experience consistent performance that displays the proper functionality.

3.3.5 Constraints

The programming must be developed using C++. The standard libraries of C++ can be used to ensure reliability and usability for the user. Also, the only operators that will be supported are $+$, $-$, $*$, $/$, $\%$, and $**$, which must use PEMDAS rules to evaluate. Initially, the input must be integers to be evaluated in the expression, but later, floating point input may be available. Finally, the command-line interface must be used for the user and system to interact.

4. Classification of Functional Requirements

Functionality	Type
Evaluate arithmetic expressions entered by the user	Essential
Apply operator precedence rules as defined	Essential
Invalid expression/Error detection and management	Essential
Specific error message display for bad input	Desirable

Arithmetic Expression Evaluator C++	Version: 1.0
Software Requirements Specifications	Date: 07/Mar/26
EECS348-SRS-TermProj-v1.0	

--	--

5. Appendices

Non-requirement Information:

System Limitations:

- Expressions must be entered using valid numeric values and supported by operators.
- Only basic arithmetic operators (+, -, *, /, %, **) are supported for this project.
- The program evaluates expressions through a command-line interface.

Supported Arithmetic Operations:

- “+” Addition
- “-” Subtraction
- “*” Multiplication
- “/” Division
- “()” Parentheses used for grouping
- “%” Modulo
- “**” Exponentiation